

S1.1: El polimorfismo estático se resuelve en tiempo de compilación, en procesos de sobrecarga de funciones y operadores. Por otro lado, el polimorfismo dinámico se resuelve en el tiempo ejecución en procesos de herencia con virtual y override.

T1.2: Porque en caso de recibir argumentos de distinto tipo la función hará una conversión automática que provocará una pérdida de datos, sin mencionar que hay funciones como void que no tienen retorno.

T1.5: En la sobrecarga la acción se repite pero de acuerdo a los datos ingresados y al retorno, en cambio con la sobrescritura se repite el formato del método de acuerdo al objeto creado.

- | | | |
|---|----------------|--|
| 1 | a Entero: 42 | b Se diferencian por el tipo y la cantidad de parámetros |
| | i Double: 3.14 | La primera recibe un int La cuarta recibe 2 int |
| | ii Texto: Hola | La segunda recibe un double |
| | iii suma: 7 | La tercera recibe un string |

c En el tiempo de compilación, según los argumentos ingresados

```
d void imprimir (bool b) {
    if (b) { cout << Bool: "Verdadero" << endl; }
    else { cout << Bool: "Falso" << endl; }
}
```

Porque C++ interpreta "Hola" como una cadena de caracteres tipo string, lo que hace necesario string() para indicar el tipo correcto.

2

a Forma :: dibujar
 Círculo :: dibujar
 Forma :: dibujar
 Forma :: dibujar

b Forma :: dibujar
 Círculo :: dibujar
 Forma :: dibujar
 Círculo :: dibujar llamada

c Porque la función probar utiliza un puntero a objeto para ejecutar el método, y al utilizar virtual se ejecuta en función del objeto

d La versión sin virtual usa ligadura estática, pues se guía por el tipo de dato, mientras que virtual usa ligadura dinámica con stable

T3.1 Convertir un puntero de clase hijo a un puntero de clase base. Es seguro porque la clase hija comparte información de la clase padre por herencia.

T3.2 Cuando un objeto hijo se copia por valor dentro de un objeto base. En sí no representa un error, pero pierde su cualidad de objeto hijo y la validez de usar sus métodos.

T3.3 Porque así se conservan las cualidades del objeto real.

T3.4 La stable es una tabla interna que guarda las direcciones de los métodos virtuales de los objetos. Se accede mediante punteros.

T3.5 Porque asegura que el contenido de las demás clases también se destruya

5 a Línea 1: Figura Línea 2: Rectángulo Línea 3: Rectángulo

b Porque se copia x como un objeto de tipo figura (según el parámetro de la función).

c El de puntero crea un puntero que apunta al valor del objeto real y el de referencia usa el objeto real directamente. Para evitar el proceso de limpieza de memoria.

d void portador (const Figura & f)

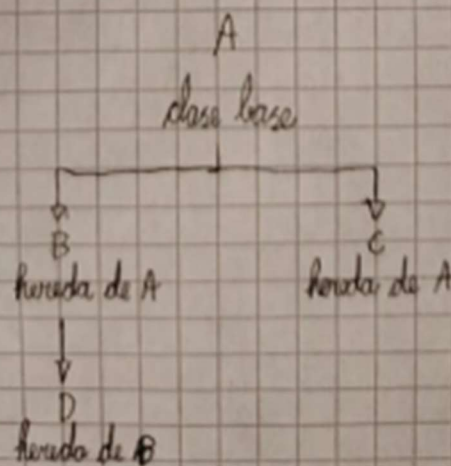
8 a Error de compilación, Animal no puede sobreescribirse así misma.
Corrección: Eliminar virtual

b Error de compilación, la función habla() es distinta a hablar()
void hablar() override

c Error de ejecución, el destructor no aplica para la clase Puro.
virtual ~Animal() {}

d No compila porque Figura tiene un método virtual puro
double area override()

11



b) [✓] caso 1: OK
[✓] caso 2: null
[✓] caso 3: OK
[✓] caso 4: OK
[✓] caso 5: null

c) [⊕] porque usa un puntero.